# Լաբորատոր աշխատանք 6
# Գրադարանի հերթագրման և տույժերի համակարգ

**Աշխատանքի նպատակը.** Գործնականում ամրապնդել C++ լեզվում օբյեկտ կողմնորոշված ծրագրավորման սկզբունքները, մասնավորապես՝ դասերի (class), մասնավոր (private) դաշտերի, հանրային (public) կոնստրուկտորների և մեթոդների կիրառումը. Սովորել կառավարել օբյեկտների զանգվածներ կամ վեկտորներ (vector), ինչպես նաև ապահովել մուտքային տվյալների ճշգրտության ստուգումը. Ծրագրի հիմնական տրամաբանությունը կառուցել բացառապես դասի մեթոդների և անհրաժեշտ օժանդակ ֆունկցիաների միջոցով.

**Խնդրի ընդհանուր պահանջները.**

- Յուրաքանչյուր խնդրի համար անհրաժեշտ է սահմանել class՝ private դաշտերով, public կոնստրուկտորով և մեթոդներով.
- Պարտադիր է օգտագործել օբյեկտների զանգված կամ vector և ապահովել մուտքային տվյալների ստուգումը.
- Հիմնական տրամաբանությունը պետք է իրականացվի դասի մեթոդների և անհրաժեշտ օժանդակ ֆունկցիաների միջոցով.
- Պատրաստի STL դասավորման գործառույթը կարելի է կիրառել միայն այն դեպքում, երբ դասախոսը թույլատրում է.

**Առաջադրանք.**

- Դասի սահմանում: Ստեղծել LibraryBook դաս, որն ունի հետևյալ դաշտերը՝ isbn, title, totalCopies, availableCopies, waitCount.
- Տվյալների բազա: Ստեղծել 12 գիրք և ընթերցողների սպասման հերթերի տվյալներ.
- Կոնստրուկտոր: Ապահովել, որ հասանելի օրինակների քանակը չգերազանցի ընդհանուր քանակը.
- borrow() մեթոդ: Տրամադրել գրքի օրինակ, իսկ դրա բացակայության դեպքում ավելացնել սպասողների քանակը.
- returnBook() մեթոդ: Վերադարձնել օրինակը և սպասողի առկայության դեպքում այն անմիջապես հատկացնել հերթի հաջորդին.
- Տույժերի հաշվարկ: Հաշվել ուշացման տույժը հետևյալ կանոնով՝ առաջին 3 օրն անվճար է, մինչև 10-րդ օրը՝ օրական 200 դրամ, իսկ 10-րդ օրվանից սկսած՝ օրական 350 դրամ.
- Վերլուծություն: Գտնել պահանջարկի ամենաբարձր ցուցանիշ ունեցող գրքերը և արտածել հերթ ունեցող գրքերի ամբողջական ցանկը.

```
#include <iostream>
#include <string>
#include <vector>
#include <limits>

class LibraryBook {
private:
std::string isbn;
std::string title;
int totalCopies;
int availableCopies;
int waitCount;

public:
LibraryBook(std::string isbn, std::string title, int totalCopies, int availableCopies,
int waitCount = 0) {
this->isbn = isbn;
this->title = title;
this->totalCopies = totalCopies;
this->availableCopies = (availableCopies <= totalCopies) ? availableCopies :
totalCopies;
this->waitCount = waitCount;
}

void borrowBook() {
if (availableCopies > 0) {
availableCopies--;
std::cout << "Book \"" << title << "\" successfully borrowed.\n";
}
else {
waitCount++;
std::cout << "Book \"" << title << "\" is out of stock. Wait list: " << waitCount <<
"\n";
}
}

void returnBook() {
if (waitCount > 0) {
waitCount--;
std::cout << "Book \"" << title << "\" returned and assigned to the next person in
line.\n";
}
else if (availableCopies < totalCopies) {
availableCopies++;
std::cout << "Book \"" << title << "\" successfully returned to the library.\n";
}
else {
std::cout << "Error: All copies are already in the library.\n";
}
}

static int calculateFine(int days) {
if (days <= 3) return 0;
if (days < 10) return (days - 3) * 200;
return 1200 + (days - 9) * 350;
}

std::string getIsbn() const { return isbn; }
std::string getTitle() const { return title; }
int getWaitCount() const { return waitCount; }
```

```
void print() const {
std::cout << "ISBN: " << isbn << " | Title: " << title
<< " | Total: " << totalCopies
<< " | Available: " << availableCopies
<< " | Waiting: " << waitCount << "\n";
}
};

int main() {
std::vector<LibraryBook> books;

int n = 0;
while (true) {
std::cout << "Enter number of books to add: ";
if (std::cin >> n && n >= 0) {
break;
}
std::cout << "Invalid input! Please enter a valid number:\n";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

for (int i = 0; i < n; ++i) {
std::string isbn, title;
int total = 0, available = 0, wait = 0;
std::cout << "\nBook " << (i + 1) << ":\n";
std::cout << "ISBN: ";
std::cin >> isbn;
std::cout << "Title: ";
std::cin.ignore();
std::getline(std::cin, title);

while (true) {
std::cout << "Total copies: ";
if (std::cin >> total && total >= 0) break;
std::cout << "Invalid input! Please enter a valid number:\n";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

while (true) {
std::cout << "Available copies: ";
if (std::cin >> available && available >= 0 && available <= total) break;
std::cout << "Invalid input! (Must not exceed total copies). Re-enter:\n";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

while (true) {
std::cout << "Wait count: ";
if (std::cin >> wait && wait >= 0) break;
std::cout << "Invalid input! Please enter a valid number:\n";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

books.push_back(LibraryBook(isbn, title, total, available, wait));
}

int choice = 0;
do {
```

```
std::cout << "\n1. Show all books\n";
std::cout << "2. Add new book\n";
std::cout << "3. Borrow book\n";
std::cout << "4. Return book\n";
std::cout << "5. Calculate fine\n";
std::cout << "6. Show highest demand books\n";
std::cout << "0. Exit\n";
std::cout << "Choose an action: ";

if (!(std::cin >> choice)) {
std::cout << "Invalid input! Please choose a number from the menu:\n";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
continue;
}

if (choice == 1) {
for (size_t i = 0; i < books.size(); ++i) {
books[i].print();
}
}
else if (choice == 2) {
std::string isbn, title;
int total = 0, available = 0;
std::cout << "ISBN: ";
std::cin >> isbn;
std::cout << "Title: ";
std::cin.ignore();
std::getline(std::cin, title);

while (true) {
std::cout << "Total copies: ";
if (std::cin >> total && total >= 0) break;
std::cout << "Invalid input! Please enter a valid number:\n";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

while (true) {
std::cout << "Available copies: ";
if (std::cin >> available && available >= 0 && available <= total) break;
std::cout << "Invalid input! Re-enter:\n";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

books.push_back(LibraryBook(isbn, title, total, available));
}
else if (choice == 3 || choice == 4) {
std::string targetIsbn;
std::cout << "Enter Book ISBN: ";
std::cin >> targetIsbn;

bool found = false;
for (auto& book : books) {
if (book.getIsbn() == targetIsbn) {
if (choice == 3) book.borrowBook();
else book.returnBook();
found = true;
break;
}
```

```
}
if (!found) {
std::cout << "Book with ISBN " << targetIsbn << " not found!\n";
}
}
else if (choice == 5) {
int days = 0;
while (true) {
std::cout << "Overdue days: ";
if (std::cin >> days && days >= 0) break;
std::cout << "Invalid input! Please enter a valid number:\n";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}
std::cout << "Fine: " << LibraryBook::calculateFine(days) << " drams\n";
}
else if (choice == 6) {
int maxWait = 0;
for (const auto& book : books) {
if (book.getWaitCount() > maxWait) {
maxWait = book.getWaitCount();
}
}
for (const auto& book : books) {
if (book.getWaitCount() == maxWait && maxWait > 0) {
std::cout << book.getTitle() << " (Waiting: " << maxWait << ")\n";
}
}
}
} while (choice != 0);

return 0;
}
```